

XML850R_R1

Proposed source, changes highlighted | SGSRC/QRPGLESRC | generated 2026-08-26 | NOT applied to the IBM i

■ added or replaced ■ removed (struck through) white = unchanged

Change IDs in this member: C4, C7, C8, C10, C11, C13, C14, C15, C16, C17, C18, C19, C20

Line numbers count the proposed source only; removed lines are unnumbered.

```
1      H Debug(*Yes) TruncNbr(*Yes) AlwNull(*UsrCtl)
2      H DftActGrp(*No) ActGrp(*Caller)
3      H Option(*SrcStmt : *NoDebugIO) ExprOpts(*ResDecPos)
4      H BndDir('SGSRC/XML850R_R1')
5      F*****
6      F*   Author           - Chris Hutchinson / Pro I.T. Resource Group, Inc. *
7      F*                   chutchinson@proitrg.com | 800-545-7630 *
8      F*                   www.proitrg.com | 336-420-0329 *
9      F*   Program Name    - XML850R_R1 *
10     F*   Program Date    - 02/06/2022 *
11     F*   Source Library   - QRPGLESRC/SGSRC *
12     F*   Object Library   - SGPGM *
13     F*   Program Descr    - Process incoming XML order transactions for *
14     F*                   Republic and OSN. ORDERS ONLY. PO changes *
15     F*                   have not been implemented yet on OSN's side. *
16     F* *
17     F*   NOTE: Elements with attributes were broken down as on page 67 *
18     F*         of Scott Klement's "Web Services and XML for RPGers.pdf" *
19     F*         at 'https://www.scottklement.com/presentations/Web%20 *
20     F*         Services%20and%20XML%20for%20RPGers.pdf' *
21     F* *
22     F*****
23     F* *
24     F*   MODIFICATIONS *
25     F* *
26     F*   02/06/2022 CDH   Modified the path handling to use the download *
27     F*                   path specified in ARPEGGIO SFTP software for *
28     F*                   the server definition defined for the retailer. *
29     F* *
30     F*   04/01/2024 CDH   Screen Graphics implemented AVATAX over the *
31     F*                   weekend. HOEUMC_W was changed to use a control *
32     F*                   file field to determine the name of the tax pgm *
33     F*                   that should be used. This routine was changed *
34     F*                   to use Harris' HOEUMC_W routine "as-is" instead *
35     F*                   of a modified version that had all the web *
36     F*                   parms removed because it would never be called *
37     F*                   directly from the web. *
38     F* *
39     F*****
40
41     * HarrisData Customer Master File
42     FHDCUST IF E K Disk
43
```

```

44      * HarrisData Item Master File
45      FHDIMST   IF   E           K Disk
46
47      * HarrisData Item Plant Master File
48      FHDIPLT   IF   E           K Disk
49
50      * HarrisData Item Warehouse Master File
51      FHDIWHS   IF   E           K Disk
52
53      * HarrisData Order Flags Master
54      FHDORFG   IF   E           K Disk
55
56      * HarrisData Ship Via Master
57      FHDSHPV   IF   E           K Disk
58
59      * HarrisData Cost Summary Master File
60      FHDMCMM   IF   E           K Disk
61
62      * HarrisData Tax Definition File
63      FHDTAXD   IF   E           K Disk
64
65      * HarrisData Order Header Work File
66      FOEHDWK   UF A E           K Disk
67
68      * HarrisData Order Detail Work File
69      FOEDTWK   O    E           Disk   Block(*NO)
70
71      * HarrisData Order Comment Work File
72      FOECMWK   UF A E           K Disk   Block(*NO)
73
74      * Arpeggio SFTP Software - Server Definition File
75      * (for download path)
76      FAFSVR1P   IF   E           K Disk
77
78      * XML Orders Header Master
79      FXMLOEHD   O    E           Disk   Block(*NO)
80
81      * XML Orders Header Master
82      FXMLOEDT   O    E           Disk   Block(*NO)
83
84      * Program Fields *
85      D fileName      s           256a
86      D ifsFile        s           256a
87      D Length         s           3s 0
88      D RestofLen      s           3s 0
89      D orderCounter    s           2s 0
90      D lineCounter     s           3s 0
91      D BuyerOrdLn     s           5s 0
92      D TempID         s           12a
93      D TempDate       s           d      DatFmt(*ISO)
94      - C10 D AddDays s 2s 0
95      + C10 D AddDays s 5s 0
96      D DaysOff        s           7s 0
97      D WorkDate       s           1a
98      D TodayISO       s           d      DatFmt(*ISO)

```

OEHDWK/H1SHTO

98	D LastDt1Date	s	d	DatFmt(*ISO)
99	D posstart	s	3s 0	
100	D X	s	5s 0	
101	D F	s	2s 0	
102	D DFL	s	1a	Dim(20)
103	D APID	s	2a	Inz('OE')
104	D HexHnd	s	64a	
105	D FGORTY	s	1a	
106	D Day_of_Week	s	1s 0	
107	D HDPCust#	s	7s 0	
108	D xmlMode	s	1a	
109	D xmlFASetID	s	10a	
110	D xmlFATransID	s	10a	
111	D OrdTotal	s	11p 2	
112	D DQ#NAM	s	10a	Inz('OEDTAQ')
113	D DQ#LIB	s	10a	Inz('*LIBL')
114	D DQ#LEN	s	5s 0	Inz(81)
115	D GiftMSG	s	256a	
116	D ShipToFnd	s	1a	Inz('N')
117	D STTxCd	s	1a	
118	D STPlaceId	s	20a	
119	D STName1	s	30a	
120	D STName2	s	30a	
121	D STNameFL	s	30a	
122	D STAdr1	s	30a	
123	D STAdr2	s	30a	
124	D STAdr3	s	30a	
125	D STCity	s	30a	
126	D STState	s	2a	
127	D STZip	s	10a	
128	D STCountry	s	3a	
129	D STEmail	s	100a	
130	D STPhone	s	15a	
131	D BillToNum	s	7s 0	
132	D BTPlaceId	s	20a	
133	D BTName1	s	30a	
134	D BTName2	s	30a	
135	D BTAdr1	s	30a	
136	D BTAdr2	s	30a	
137	D BTAdr3	s	30a	
138	D BTCity	s	30a	
139	D BTState	s	2a	
140	D BTZip	s	10a	
141	D BTCntry	s	3a	
142	D BTEmail	s	100a	
143	D BTPhone	s	15a	
144	D CSPlaceId	s	20a	
145	D CSName1	s	30a	
146	D CSName2	s	30a	
147	D CSAdr1	s	30a	
148	D CSAdr2	s	30a	
149	D CSAdr3	s	30a	
150	D CSCity	s	30a	
151	D CSState	s	2a	
152	D CSZip	s	10a	

153		D CSEmail	s	100a	
154		D CSPhone	s	15a	
155		D IVPlaceId	s	20a	
156		D IVName1	s	30a	
157		D IVName2	s	30a	
158		D IVAdr1	s	30a	
159		D IVAdr2	s	30a	
160		D IVAdr3	s	30a	
161		D IVCity	s	30a	
162		D IVState	s	2a	
163		D IVZip	s	10a	
164		D IVEmail	s	100a	
165		D IVPhone	s	15a	
166		D UsePath	s	15a	
167		D Cmd	s	500a	
168		D CmdLen	s	15p 5	
169		D DropShip	s	7s 0	
170		D FAPath	s	155a	Varying
171		D FAFile	s	100a	Varying
172		D FAFull	s	256a	Varying
173		D RightNow	s	Z	
174		D Avail	s	13p 4	
175		D from	s	100a	
176		D To	s	100a	
177		D replyTo	s	100a	
178		D subject	s	100a	
179		D text	s	9999a	
180		D distribution	s	100a	
181		D email	s	0	class(*java:RPGMail)
182		D Alert	s	10a	
183		D AlertSubj	s	200a	
184		D AlertMsg	s	9999a	
185		D pass	s	10i 0	
186		D StockItem	s	1a	
187		D orderAction	s	10a	
188		D CMSVSV	s	2a	
189		D NullInd	s	2b 0	
190		D UDCDays	s	2s 0	
191	+	C10	D WkClass	s	Like(IMPCLS)
192	+	C10	D CustClass	s	Like(CMCCLS)
193	+	C10	D LeadDays	s	5s 1
194	+	C10	D ReqDays	s	5s 0
195	+	C10	D ProcessedItems	s	3s 0
196					
197		D XMLorderId	s	20a	
198		D LastOrder	s	8s 0	
199		D CurDate	s	D	DatFmt(*ISO)
200		D CurTime	s	T	
201		D Transact	s	10a	
202		D HWLinCnt	s	5s 0	
203		D lineTotal	s	10i 0	
204		D OrderTotal	s	10i 0	
205		D CustomItem	s	1a	
206		D DebugFlag	s	1a	
207		D SaveDesc	s	300a	

208	D CharPos	s	3s 0	
209				
210	D 01IMDS	s	30a	
211	D 01BITEM	s	35a	
212	D 01VITEM	s	35a	
213	D 01ITEM	s	15a	
214	D 01CUOM	s	3a	Inz('EA')
215	D 01PUOM	s	3a	Inz('EA')
216	D 01QUOM	s	3a	Inz('EA')
217	D 01ORL#	s	3P 0	
218	D 01QORD	s	13p 4	
219	D 01RQDTA	s	25a	
220	D 01RQDTD	s	D	DatFmt(*ISO-)
221	D 01COST	s	13p 5	
222	D 01SLPR	s	13p 5	
223	D 01SLPRXLT	s	13a	
224	D 01SVSV	s	2a	
225	D 01CNCDT	s	8a	
226	D 01DLVDT	s	8a	
227				
228	D ErrFlag	s	1a	
229	D ErrDescr	s	100a	
230	D RecsWrtn	s	1a	
231				
232				
233	D HWPO#	s	22a	OEHDWK/H1ORRF
234	D HWOrdDate	s	25a	yyyymmdd
235	D HWMerchDpt	s	10a	
236	D HWShpToID	s	12a	OEHDWK/H1SHTO
237	D HWBillToID	s	12a	OEHDWK/H1BLTO
238	D HWLineCnt	s	3s 0	
239	D HWCustID	s	12a	OEHDWK/H1BLTO
240	D HWInvToID	s	12a	OEHDWK/H1BLTO
241	D HWShipCd	s	15a	OEHDWK/H1SVSV-H1SVDS
242	D HWControl	s	5a	
243	D HWSlsDiv	s	10a	
244	D HWCustOrd	s	15a	???????
245	D HWBuyCtrct	s	10a	???????
246	D HWReqShpD	s	10a	
247	D HWPOType	s	2a	
248	D HWGiftMSG	s	256a	
249				
250	D xmLOCTL	s	8p 0	
251	D xmLFRT	s	9p 2	
252	D xmLCHRG	s	9p 2	
253	D xmLUPDTOT	s	1a	
254				
255	D CustOrdLn	s	3s 0	
256	D CustRqDt	s	7s 0	
257	D CustDesc	s	30a	
258	D CustCmt	s	60a	
259	D CustQOrd	s	13s 4	
260	D CustSlPr	s	13s 5	
261				
262	* XML Acknowledgement Variables *			

```

263
264 * Customer#
265 D XMLACKCUST      s          20A
266 * Acknowledgement Date (YYYY-MM-DD)
267 D XMLACKDATE      s          D    datfmt(*ISO)
268 * Ack. Order ID
269 D XMLACKORDR      s          20A
270 * Ack. Customer PO#
271 D XMLCUSTPO       s          22A
272 * Ack. Type
273 D XMLACKTYPE      s          6A
274 * Customer PO Date
275 D XMLCUSTPD       s          D    datfmt(*ISO)
276
277
278 D HOEUMC_W        PR          ExtPgm('HOEUMC_W')
279 D HexHnd          64a
280 D xmlDBID         2a
281 D xmlOCTL         8p 0
282 D xmlFRT          9p 2
283 D xmlCHRG         9p 2
284 D xmlUPDTOT       1a
285
286 D Ack             PR          ExtPgm('XMLACKR')
287 D xmlRetailer     15a
288 D HWPO#          22a
289 D ErrFlag        1a
290 D ErrDescr       100a Options(*NoPass)
291
292 D XML850R_R1      PR          ExtPgm('XML850R_R1')
293 D xmlDBID         2a
294 D xmlRetailer     15a
295 D xmlFile        100a
296 D xmlSvrDfn      10a
297
298 D XML850R_R1      PI
299 D xmlDBID         2a
300 D xmlRetailer     15a
301 D xmlFile        100a
302 D xmlSvrDfn      10a
303
304 D error           DS          LikeDS(Error_Info)
305
306 D DTAQ            E DS          ExtName(OEDTAQPL)
307
308
309 * Prototype for Send Data Queue (QSNDDTAQ) API
310 D QSNDDTAQ        PR          ExtPgm('QSNDDTAQ')
311 D DQ#NAM          10a Const
312 D DQ#LIB          10a Const
313 D DQ#LEN          5p 0 Const
314 D Data            32766a Options(*varsize)
315
316 * QCmdExc Parameters *
317 D ExecCmd         PR          EXTPGM('QCMDEXC')

```

```

318 D Cmd 500a
319 D CmdLen 15p 5
320
321 * Prototypes for RPGMail *
322 /copy RPGMail/qsource,RPGMailCP
323 /copy RPGMail/qsource>ErrorCP
324
325 // QMHSNDPM prototype + work fields - log alert send failures (3c)
326 dcl-pr SndPgmMsg ExtPgm('QMHSNDPM');
327     p_MsgID      char(7)      const;
328     p_MsgFile     char(20)     const;
329     p_MsgData     char(32767)  const options(*varsize);
330     p_MsgDataLen  int(10)      const;
331     p_MsgType     char(10)     const;
332     p_CallStkEnt  char(10)     const;
333     p_CallStkCnt  int(10)      const;
334     p_MsgKey      char(4);
335     p_ErrCode     char(32767)  options(*varsize);
336 end-pr;
337
338 dcl-s MsgKey char(4);
339 dcl-s LogTxt char(512);
340 dcl-ds ApiErr qualified;
341     BytesProv int(10) inz(8);
342     BytesAvail int(10) inz(0);
343 end-ds;
344
345
346
347
348
349 *****
350 ** XML Data TEMPLATES **
351 *****
352
353 D*-----
354 D*| Header Template |
355 D*-----
356
357 dcl-ds Sender_T Qualified Template ;
358     dcl-ds Credential likeDS(Credential_T) ;
359     UserAgent Char(20) ;
360 // end-ds ;      ---> NOT specified after a DS is declared with likeDS!!!
361 end-ds Sender_T ;
362
363 dcl-ds Credential_T Qualified Template ;
364     domain      Char(20) ;
365     Identity     Char(20) ;
366     SharedSecret Char(15) ;
367 end-ds Credential_T ;
368
369 *****
370 * XML Data Structure Definitions *
371 *****
372 dcl-ds cXML Qualified ;

```

```

373     version      Char(10) ;
374     payloadID    Char(50) ;
375
376     dcl-ds Header ;
377
378         dcl-ds From ;
379             dcl-ds Credential ;
380                 domain Char(20) ;
381                 Identity Char(20) ;
382             end-ds Credential ;
383         end-ds From ;
384
385         dcl-ds To ;
386             dcl-ds Credential ;
387                 domain Char(20) ;
388                 Identity Char(20) ;
389             end-ds Credential ;
390         end-ds To ;
391
392         dcl-ds Sender ;
393             dcl-ds Credential likeDS(Credential_T) inz(*likeDS) ;
394         //     end-ds Credential ;           ---> Not used on a data structure declared with likeDS!!!
395         end-ds Sender ;
396         UserAgent Char(50);
397     end-ds Header ;
398
399     dcl-ds Request ;
400         // deploymentMode Char(10) ;
401
402         dcl-ds OrderRequest Dim(10);
403             dcl-ds OrderRequestHeader ;
404                 orderID Char(15) ;
405                 numoforderID Int(10);
406                 orderDate Char(25) ;
407                 orderType Char(8) ;
408                 orderVersion Zoned(2:0) ;
409                 type Char(10) ;
410             dcl-ds Total ;
411                 dcl-ds Money ;
412                     alternateAmount Char(10) ;           // Was Zoned(8:2) but they don't send it
413                     alternateCurrency Char(10) ;
414                     currency Char(3) ;
415                     dsvalue Char(15) ;
416                 end-ds Money ;
417             end-ds Total ;
418             dcl-ds ShipTo ;
419                 dcl-ds Address ;
420                     isoCountryCode Char(5) ;
421                     addressID Char(12) ;
422                 dcl-ds Name ;
423                     xml_lang Char(2);           // Was xml:lang; Works w/case convert
424                     dsvalue Char(50);
425                 end-ds Name ;
426             dcl-ds PostalAddress ;
427                 name Char(10) ;

```



```

428         DeliverTo Char(30) Dim(2) ;
429         Street Char(30) Dim(4) ;
430         City Char(15) ;
431         State Char(2) ;
432         PostalCode Char(10) ;
433         dcl-ds Country ;
434             isoCountryCode Char(2) ;
435             dsvalue Char(50) ;
436         end-ds Country ;
437     end-ds PostalAddress ;
438     Email Char(50) ;
439     dcl-ds Phone ;
440         name Char(10) ;
441         dcl-ds TelephoneNumber ;
442             dcl-ds CountryCode ;
443                 isoCountryCode Char(2) ;
444                 dsvalue Char(1) ;
445             end-ds CountryCode ;
446             AreaOrCityCode Char(3) ;
447             Number Char(10) ;
448             Extension Char(6) ;
449         end-ds TelephoneNumber ;
450     end-ds Phone ;
451     dcl-ds Fax ;
452         name Char(10) ;
453         dcl-ds TelephoneNumber ;
454             dcl-ds CountryCode ;
455                 isoCountryCode Char(2) ;
456                 dsvalue Char(1) ;
457             end-ds CountryCode ;
458             AreaOrCityCode Char(3) ;
459             Number Char(10) ;
460             Extension Char(6) ;
461         end-ds TelephoneNumber ;
462     end-ds Fax ;
463 end-ds Address;
464 end-ds ShipTo;
465
466 dcl-ds BillTo ;
467     dcl-ds Address ;
468         isoCountryCode Char(2) ;
469         addressID Char(12) ;
470         dcl-ds Name ;
471             xml_lang Char(2) ;
472             dsvalue Char(32) ;
473         end-ds Name ;
474         DelverTo Char(50) ;
475         dcl-ds PostalAddress ;
476             name Char(30) ;
477             Street Char(50) Dim(4) ;
478             City Char(30) ;
479             State Char(2) ;
480             PostalCode Char(10) ;
481             dcl-ds Country ;
482                 isoCountryCode Char(2) ;

```

```

483         dsvalue          Char(25) ;
484     end-ds Country ;
485 end-ds PostalAddress ;
486 Email          Char(50) ;
487 dcl-ds Phone ;
488     name        Char(10);
489     dcl-ds TelephoneNumber ;
490         dsvalue          Char(25) ;
491         isoCountryCode   Char(2) ;
492         dsvalue          Char(1) ;
493     end-ds CountryCode ;
494     AreaOrCityCode   Char(3) ;
495     Number           Char(10) ;
496     Extension        Char(6) ;
497     end-ds TelephoneNumber ;
498 end-ds Phone ;
499 dcl-ds Fax ;
500     name        Char(10) ;
501     dcl-ds TelephoneNumber ;
502         dsvalue          Char(25) ;
503         isoCountryCode   Char(2) ;
504         dsvalue          Char(1) ;
505     end-ds CountryCode ;
506     AreaOrCityCode   Char(3) ;
507     Number           Char(10) ;
508     Extension        Char(6) ;
509     end-ds TelephoneNumber ;
510 end-ds Fax ;
511 end-ds Address ;
512 end-ds BillTo ;
513
514 dcl-ds Shipping ;
515     trackingDomain    Char(15) ;
516     dcl-ds Money ;
517         currency      Char(3) ;
518         dsValue       Char(12) ;
519 //         dsValue     Zoned(7:2) ;
520     end-ds Money ;
521     dcl-ds Description ;
522         xml_lang      Char(2) ;
523         dsvalue       Char(30);
524     end-ds Description ;
525 end-ds Shipping ;
526
527 dcl-ds Contact ;
528     role              Char(10) ;
529     dcl-ds Name ;
530         xml_lang      Char(2) ;
531         dsValue       Char(30) ;
532     end-ds Name ;
533     dcl-ds PostalAddress ;
534         name          Char(30) ;
535         Street         Char(50) Dim(4) ;
536         City           Char(30) ;
537         State          Char(2) ;

```

```

538         PostalCode    Char(10) ;
539         dcl-ds Country ;
540         isoCountryCode Char(2) ;
541         dsvalue        Char(25) ;
542         end-ds Country ;
543     end-ds PostalAddress ;
544     Email              Char(50) ;
545     dcl-ds Phone ;
546         name          Char(10);
547         dcl-ds TelephoneNumber ;
548             dcl-ds CountryCode ;
549                 isoCountryCode Char(2) ;
550                 dsvalue        Char(1) ;
551             end-ds CountryCode ;
552             AreaOrCityCode Char(3) ;
553             Number         Char(10) ;
554             Extension      Char(6) ;
555             AreaOrCityCode Zoned(3:0) ;
556             Number         Zoned (10 : 0) ;
557             Extension      Zoned(6 : 0) ;
558         end-ds TelephoneNumber ;
559     end-ds Phone ;
560 end-ds Contact ;
561 Comments    Char(150) ;
562 dcl-ds Extrinsic Dim(23) ;
563     name     Char(15) ;
564     dsValue Char(50) ;
565 end-ds Extrinsic;
566 end-ds OrderRequestHeader ;
567
568 dcl-ds ItemOut Dim(420) ;
569     lineNumber      Zoned(5:0) ;
570     quantity        Char(13) ;
571     requestedDeliveryDate Char(25) ;
572     dcl-ds ItemID ;
573         SupplierPartID Char(15) ;
574         SupplierPartAuxiliaryID Char(15) ;
575     end-ds ItemID ;
576
577     dcl-ds ItemDetail ;
578         dcl-ds UnitPrice ;
579             dcl-ds Money ;
580                 currency      Char(3) ;
581                 dsvalue       Char(13) ;
582             end-ds Money ;
583         end-ds UnitPrice ;
584         dcl-ds Description ;
585             xml_lang Char(2) ;
586             dsvalue Char(300) ;
587         end-ds Description;
588         UnitOfMeasure Char(10) ;
589         dcl-ds Classification ;
590             domain Char(10) ;
591             Classification Char(10) ;
592         end-ds Classification ;

```

```

593         dcl-ds Extrinsic Dim(36) ;
594         name      Char(15) ;
595         dsValue    Char(50) ;
596     end-ds Extrinsic ;
597 end-ds ItemDetail ;
598
599 dcl-ds Supplier ;
600 dcl-ds Name ;
601     xml_lang      Char(2) ;
602     dsValue        Char(30) ;
603 end-ds Name ;
604 Comments          Char(150) ;
605 dcl-ds SupplierID ;
606     domain         Char(30) ;
607     dsValue         Zoned(15:0) ;
608 dcl-ds SupplierLocation ;
609     dcl-ds Address ;
610         dcl-ds Name ;
611             xml_lang      Char(2) ;
612             dsValue        Char(10) ;
613         end-ds Name ;
614         dcl-ds PostalAddress ;
615             DeliverTo      Char(30) ;
616             Street          Char(30) Dim(4) ;
617             City            Char(30) ;
618             State           Char(2) ;
619             PostalCode      Char(10) ;
620         dcl-ds Country ;
621             isoCountryCode  Char(2) ;
622             dsValue         Char(2) ;
623         end-ds Country ;
624     end-ds PostalAddress ;
625     Email           Char(30) ;
626 dcl-ds Phone ;
627     dcl-ds TelephoneNumber ;
628         dcl-ds CountryCode ;
629             isoCountryCode  Char(2) ;
630             dsValue         Char(2) ;
631         end-ds CountryCode ;
632         Number           Zoned(11:0) ;
633         Extension        Zoned(6:0) ;
634         AreaOrCityCode   Zoned(3:0) ;
635         AreaOrCityCode   Char(3) ;
636         Number           Char(8) ;
637         Extension        Char(6) ;
638     end-ds TelephoneNumber ;
639 end-ds Phone ;
640 dcl-ds Fax ;
641     dcl-ds TelephoneNumber ;
642         dcl-ds CountryCode ;
643             isoCountryCode  Char(2) ;
644             dsValue         Char(2) ;
645         end-ds CountryCode ;
646         AreaOrCityCode   Zoned(3:0) ;
647         Number           Zoned(11:0) ;

```

```

648      //          Extension      Zoned(6:0) ;
649          AreaOrCityCode      Char(3) ;
650          Number              Char(7) ;
651          Extension          Char(6) ;
652      end-ds TelephoneNumber ;
653      end-ds Fax ;
654      end-ds Address ;
655      dcl-ds OrderMethods ;
656          dcl-ds OrderMethod ;
657              dcl-ds OrderTarget ;
658                  OtherOrderTarget Char(30) ;
659              end-ds OrderTarget ;
660              OrderProtocol Char(30) ;
661          end-ds OrderMethod ;
662          dcl-ds Contact ;
663              dcl-ds Name ;
664                  xml_lang Char(2) ;
665                  dsValue Char(30) ;
666              end-ds Name ;
667              Email Char(30) ;
668          dcl-ds Phone ;
669              dcl-ds TelephoneNumber ;
670                  dcl-ds CountryCode ;
671                      isoCountryCode Char(2) ;
672                      dsValue Char(2) ;
673                  end-ds CountryCode ;
674          //          AreaOrCityCode Char(15) ;
675          //          Number Zoned(11:0) ;
676          //          Extension Zoned(6:0) ;
677              AreaOrCityCode Char(3) ;
678              Number Char(7) ;
679              Extension Char(6) ;
680          end-ds TelephoneNumber ;
681      end-ds Phone ;
682      end-ds Contact ;
683      end-ds OrderMethods ;
684      end-ds SupplierLocation ;
685      end-ds SupplierID ;
686      end-ds Supplier ;
687      dcl-ds ShipTo ;
688          dcl-ds Address ;
689              isocountryCode Char(2) ;
690              AddressID Char(12) ;
691          dcl-ds Name ;
692              xml_lang Char(2) ;
693              dsValue Char(30) ;
694          end-ds Name ;
695          dcl-ds PostalAddress ;
696              DeliverTo Char(30) ;
697              Street Char(30) Dim(4) ;
698              City Char(30) ;
699              State Char(2) ;
700              PostalCode Char(10) ;
701          dcl-ds Country ;
702              isoCountryCode Char(2) ;

```

```

703         dsValue          Char(2) ;
704     end-ds Country ;
705 end-ds PostalAddress ;
706 Email    Char(30) ;
707 dcl-ds Phone ;
708     dcl-ds TelephoneNumber ;
709     dcl-ds CountryCode ;
710         isoCountryCode Char(2) ;
711         dsValue          Char(2) ;
712     end-ds CountryCode ;
713 //     AreaOrCityCode    Zoned(3:0) ;
714 //     Number              Zoned(11:0) ;
715 //     Extension           Zoned(6:0) ;
716     AreaOrCityCode    Char(3) ;
717     Number             Char(8) ;
718     Extension          Char(6) ;
719 end-ds TelephoneNumber ;
720 end-ds Phone ;
721 end-ds Address ;
722 end-ds ShipTo ;
723 dcl-ds Tax ;
724     dcl-ds Money ;
725         currency Char(3) ;
726         dsValue Char(12) ;
727     end-ds Money ;
728     dcl-ds Description ;
729         xml_lang Char(2) ;
730         dsValue Char(50) ;
731     end-ds Description ;
732     dcl-ds TaxDetail ;
733         category Char(11) ;
734         purpose Char(25) ;
735     dcl-ds TaxAmount ;
736         dcl-ds Money ;
737             currency Char(3) ;
738             dsValue Char(12) ;
739         end-ds Money ;
740     end-ds TaxAmount ;
741     dcl-ds TaxLocation ;
742         xml_lang Char(2) ;
743         dsValue Char(20) ;
744     end-ds TaxLocation ;
745     dcl-ds Description ;
746         xml_lang Char(2) ;
747         dsValue Char(50) ;
748     end-ds Description ;
749 end-ds TaxDetail ;
750 end-ds Tax ;
751 dcl-ds Comments ;
752     xml_lang Char(2) ;
753     dsValue Char(50) ;
754 end-ds Comments ;
755 end-ds ItemOut ;
756 end-ds OrderRequest ;
757 end-ds Request ;

```

```

758         end-ds cXML ;
759
760
761
762         * Constants *
763         D Up          C          'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
764         D Lo          C          'abcdefghijklmnopqrstuvwxyz'
765         D CrLf        C          Const(x'0d25')
766
767         * Search/Replace Constants
768         - E19 D apos      C          'andapos;'
769         + C19         D apos      C          '&apos;'
770         - E19 D quot      C          'andquot;'
771         + C19         D quot      C          '&quot;'
772         - E19 D amper     C          'andamp;'
773         + C19         D amper     C          '&amp;'
774         - E19 D lt         C          'andlt;'
775         + C19         D lt        C          '&lt;'
776         - E19 D gt         C          'andgt;'
777         + C19         D gt        C          '&gt;'
778
779
780
781         * ----- Main Module -----*
782
783         /Free
784
785         ErrFlag = 'N';
786
787         // Prepare email for alerts to Customer Service...
788         Monitor;
789         RPGMail_init();
790         On-error;
791         error = Error_catch();
792         EndMon;
793
794
795         Exsr getXMLData;
796
797         // Process UP TO 10 orders per cXML file (per the "Dim"
798         // statement on the SGOrder Data Structure spec)
799         Exsr loadDB2Data;
800
801         Exsr exitProgram;
802
803         *InLR = *On;
804
805         //-----*
806         // getXMLData Subroutine will pull all XML variables out and *
807         // put them into their appropriate production data fields *
808         //-----*
809
810         BegSr getXMLData;
811
812         Select;

```

```

808     When XMLRetailer = 'amerigas';
809         BillToNum = 1590000;
810     When XMLRetailer = 'republic';
811         BillToNum = 1000;
812     When XMLRetailer = 'wastemgmt';
813         BillToNum = 5000;
814     EndSL;
815
816     usepath = XMLRetailer;
817
818     // Pull extraneous data required based on the trading
819     // partner being processed...
820     Exec SQL
821         Select Substr(UDCDESC3,1,1), Dec(UDCDESC2,7,0)
822             Into :xmlMode, :HDPCust#
823             From ProITRG/UDCDETAIL
824             Where UDCSYSTEMD = 'XML' And UDCKEY = :XMLRetailer;
825
826     If SQLCod <> 0;
827         // Notify somebody of an error trying to process :XMLRetailer!!!
828         // Don't know how this could happen if the user got this far.
829         *InLR = *On;
830         Return;
831     EndIf;
832
833     // Pull the path used for the download to extract the filename
834     Chain xmlSvrDfn AFSVR1P ;
835
836     Length = %len(%Trim(SFDWNFLR)) ;
837     If Length = 0 ;
838         C11 Alert = 'XMLORDERS';
839         + C11 Alert = 'XMLORDERR';
840         AlertSubj = %Trim(AlertSubj) + ' ' +
841             'Republic OSN Order Alert: ' + %Trim(xmlRetailer) +
842             + C14 'Republic OSN Order Alert: ' + %Trim(xmlRetailer) ;
843         AlertMsg = fileName;
844
845         AlertMsg = %Trim(AlertMsg) +
846             'Details: Order cannot be processed. ' +
847             'No path in the ' + %Trim(xmlSvrDfn) +
848             ' server definition in ARPSFTP.' ;
849         Exsr Notify ;
850         *InLR = *On ;
851         Return ;
852     EndIf;
853
854     Select;
855     When xmlMode = 'T';
856         filename = '/orders/archive inbound/' +
857             %Trim(xmlRetailer) + '/orders/' +
858             %Trim(xmlFile) ;
859
860

```



```

861
862 When xmlMode = 'L';
863 RestofLen = %len(xmlFile) - Length;
864 ifsFile = %Subst(xmlFile:Length + 1:RestofLen);
865 filename = '/orders/inbound/' +
866           %Trim(xmlRetailer) + '/' + %Trim(ifsFile);
867           // %Trim(xmlRetailer) + '/' + %Trim(xmlFile); *** Used for TEST ONLY
868
869 EndSL;
870
871
872 //-----*
873 // Original Code. XML-INTO can't handle the "NIL" variables so this *
874 // was changed to a DATA-INTO opcode with the XMLNSC specified which *
875 // CAN handle the "NIL" element values. (ex. <AreaOrCityCode/>) *
876 //-----*
877 // *
878 // XML-INTO(EH) cXML %xml(%Trim(fileName): *
879 // 'case=convert doc=file datasubf=dsvalue + *
880 // allowextra=yes allowmissing=yes + *
881 // countprefix=numof ns=merge'); *
882 // *
883 //-----*
884
885 // DATA-INTO cXML %Data(fileName : *
886 XML-INTO cXML %XML(%Trim(fileName) :
887 'case=convert doc=file datasubf=dsvalue +
888 allowextra=yes allowmissing=yes +
889 countprefix=numof ns=merge') ;
890 // %Parser(XMLNSC); *
891
892 EndSr;
893
894 //-----*
895 // loadDB2Data subroutine will populate all the data required *
896 // for HarrisDatas' normal order processing. *
897 //-----*
898
899 BegSr loadDB2Data;
900
901 orderTotal = cXML.Request.OrderRequest(1).
902             OrderRequestHeader.numofOrderID;
903
904 For orderCounter = 1 to orderTotal;
905
906 // Check to see if this is a Change or Cancel request.
907 // If it is, just email customer service and don't
908 // process the order.
909 If %Trim(cXML.Request.OrderRequest(orderCounter).
910 OrderRequestHeader.orderType) = 'regular' AND
911 %Trim(cXML.Request.OrderRequest(orderCounter).
912 OrderRequestHeader.type) = 'new';
913
914 Exsr writeOrder;
915

```

```

916 Else;
917
918 // Notify Customer Service of Order Type we
919 // can't handle...
920
921 orderAction = cXML.Request.OrderRequest(orderCounter).
922             OrderRequestHeader.type;
923
924 // Convert orderAction to uppercase with SQL
925 Exec SQL
926     Set :orderAction = Upper(:orderAction);
927
928 Alert = 'ORDUPDATE';
929 AlertSubj = %Trim(AlertSubj) + ' ' +
930            'Ariba Order Alert: ' + %Trim(xmlRetailer) +
931            ' - ORDER ' + %Trim(orderAction) +
932            ' - PO# ' + %Trim(%Char(cXML.Request.
933            OrderRequest(orderCounter).
934            OrderRequestHeader.orderID));
935 AlertMsg = fileName;
936
937 // AlertMsg = %Trim(AlertMsg) + 'Ariba Order: '
938 //           + HWPO# + CrLf;
939 // AlertMsg = %Trim(AlertMsg) + 'Order Line: '
940 //           + %Char(010rL#) + CrLf;
941 // AlertMsg = %Trim(AlertMsg) + 'Item: '
942 //           + 01BItem + CrLf;
943 // AlertMsg = %Trim(AlertMsg) + 'Issue: Configuration'
944 //           + CrLf;
945 // AlertMsg = %Trim(AlertMsg) +
946 //           'Details: Stock item was not found ' +
947 //           'on the HarrisData Item Plant file.';
948 Exsr Notify;
949
950 EndIf;
951
952 If ErrFlag = 'Y';
953     Exsr Notify;
954 EndIf;
955
956 EndFor;
957
958 EndSr;
959 //-----*
960
961
962
963 //-----*
964 /* writeOrder loads the fields for the transaction header
965 //-----*
966 BegSr writeOrder;
967
968 Clear OEHDWKR;
969 Clear XMLOEHDR;
970

```

```

971 // Write the header for the order...
972 XMLOrderID = cXML.Request.OrderRequest(orderCounter).
973             OrderRequestHeader.orderID;
974
975 Exsr NxtHDOrd;
976
977 HWPO# = cXML.Request.
978         OrderRequest(orderCounter).
979         OrderRequestHeader.orderID;
980 HWPO# = %XLate(Lo:Up:HWPO#);
981
982 - C18 ----- If HWPO# = '410022893';
983 - C18 ----- DebugFlag = 'Y';
984 - C18 ----- EndIf;
985 + C18 // C18 - leftover debug, DebugFlag is never read
986 + C18 // If HWPO# = '410022893';
987 + C18 // DebugFlag = 'Y';
988 + C18 // EndIf;
989
990 HWOOrdDate = %Char(%Date(%Subst(cXML.Request.
991                               OrderRequest(orderCounter).
992                               OrderRequestHeader.orderDate:1:10):*ISO-));
993
994 HWShpToID = cXML.Request.
995             OrderRequest(orderCounter).
996             OrderRequestHeader.ShipTo.
997             Address.addressID;
998
999 If cXML.Request.
1000    OrderRequest(orderCounter).
1001    OrderRequestHeader.ShipTo.
1002    Address.PostalAddress.
1003    DeliverTo(1) <> *Blanks;
1004    STName1 = %Trim(cXML.Request.
1005                  OrderRequest(orderCounter).
1006                  OrderRequestHeader.ShipTo.Address.
1007                  PostalAddress.DeliverTo(1));
1008 Else;
1009    STName1 = %Trim(cXML.Request.
1010                  OrderRequest(orderCounter).
1011                  OrderRequestHeader.ShipTo.Address.
1012                  Name.dsvalue);
1013 EndIf;
1014
1015 STName1 = %XLate(Lo:Up:STName1);
1016 STAdr1 = cXML.Request.
1017         OrderRequest(orderCounter).
1018         OrderRequestHeader.ShipTo.
1019         Address.PostalAddress.DeliverTo(2);
1020 STAdr1 = %XLate(Lo:Up:STAdr1);
1021 STAdr2 = cXML.Request.
1022         OrderRequest(orderCounter).
1023         OrderRequestHeader.ShipTo.
1024         Address.PostalAddress.Street(1);
1025 STAdr2 = %XLate(Lo:Up:STAdr2);
1026 STAdr3 = cXML.Request.
1027         OrderRequest(orderCounter).

```

```

1023         OrderRequestHeader.ShipTo.
1024         Address.PostalAddress.Street(2);
1025     STAdr3    = %XLate(Lo:Up:STAdr3);
1026     STAdr3 = *Blanks;
1027     STCity    = cXML.Request.
1028         OrderRequest(orderCounter).
1029         OrderRequestHeader.ShipTo.
1030         Address.PostalAddress.City;
1031     STCity    = %XLate(Lo:Up:STCity);
1032     STState    = cXML.Request.
1033         OrderRequest(orderCounter).
1034         OrderRequestHeader.ShipTo.
1035         Address.PostalAddress.State;
1036     STState    = %XLate(Lo:Up:STState);
1037     STZip      = cXML.Request.
1038         OrderRequest(orderCounter).
1039         OrderRequestHeader.ShipTo.
1040         Address.PostalAddress.PostalCode;
1041     STCountry  = cXML.Request.
1042         OrderRequest(orderCounter).
1043         OrderRequestHeader.ShipTo.
1044         Address.PostalAddress.
1045         Country.isoCountryCode;
1046     STCountry  = %XLate(Lo:Up:STCountry);
1047
1048     Monitor;
1049     // All ShipTos won't provide a phone number
1050
1051     STPhone    = %Char(cXML.Request.
1052         OrderRequest(orderCounter).
1053         OrderRequestHeader.ShipTo.
1054         Address.Phone.TelephoneNumber.
1055         AreaOrCityCode) +
1056         %Char(cXML.Request.
1057         OrderRequest(orderCounter).
1058         OrderRequestHeader.ShipTo.
1059         Address.Phone.TelephoneNumber.
1060         Number);
1061
1062     On-Error;
1063     STPhone = *Blanks;
1064     EndMon;
1065
1066     BTName1    = cXML.Request.
1067         OrderRequest(orderCounter).
1068         OrderRequestHeader.BillTo.
1069         Address.Name.dsvalue;
1070     BTName1    = %XLate(Lo:Up:BTName1);
1071     BTAdr1     = cXML.Request.
1072         OrderRequest(orderCounter).
1073         OrderRequestHeader.BillTo.
1074         Address.PostalAddress.Street(1);
1075     BTAdr1     = %XLate(Lo:Up:BTAdr1);
1076     BTAdr2     = cXML.Request.
1077         OrderRequest(orderCounter).
1078         OrderRequestHeader.BillTo.

```

```

1078         Address.PostalAddress.Street(2);
1079     BTAdr2 = %XLate(Lo:Up:BTAdr2);
1080     BTCity = cXML.Request.
1081         OrderRequest(orderCounter).
1082         OrderRequestHeader.BillTo.
1083         Address.PostalAddress.City;
1084     BTCity = %XLate(Lo:Up:BTCity);
1085     BTState = cXML.Request.
1086         OrderRequest(orderCounter).
1087         OrderRequestHeader.BillTo.
1088         Address.PostalAddress.State;
1089     BTState = %XLate(Lo:Up:BTState);
1090     BTZip = cXML.Request.
1091         OrderRequest(orderCounter).
1092         OrderRequestHeader.BillTo.
1093         Address.PostalAddress.PostalCode;
1094     BTCntry = cXML.Request.
1095         OrderRequest(orderCounter).
1096         OrderRequestHeader.BillTo.
1097         Address.PostalAddress.
1098         Country.isoCountryCode;
1099     BTCntry = %XLate(Lo:Up:STCountry);
1100
1101     // Determine the ShipTo as follows...
1102     // (Preferred)
1103     // - Pull the Customer whose addressID in the cXML document
1104     //   that matches the CMUDF4 field on HDCUST and has a BillTo
1105     //   of :BillToNum
1106     // (Secondary option)
1107     // - If that doesn't exist, pull the Customer whose addressID
1108     //   in the cXML document matches the CMUDF4 field on HDCUST
1109     //   and use that BillTo and ShipTo
1110     // (Tertiary option)
1111     // - If neither of those exist, set up the order as a drop
1112     //   ship with the information provided in the cXML document
1113     //   and notify orders@screen-graphics.com of the issue.
1114
1115     ShipToFnd = 'N';
1116
1117     Exec SQL
1118     Select CMCUST, CMSV, CMCTXC Into :H1ShTo, :CMSVSV, :STTXCD
1119     From HDCUST
1120     Where CMUDF4 = :HWSHpToID
1121     And CMBLTO = :BillToNum;
1122
1123     If SQLCod = 0;
1124         ShipToFnd = 'Y';
1125     EndIf;
1126
1127     If ShipToFnd = 'N';
1128         Exec SQL
1129         Select CMCUST, CMBLTO, CMSV, CMCTXC
1130         Into :H1ShTo, :H1B1To, :CMSVSV, :STTXCD
1131         From HDCUST
1132         Where CMUDF4 = :HWSHpToID;

```

```

1133     If SQLCod = 0;
1134         ShipToFnd = 'Y';
1135     EndIf;
1136 EndIf;
1137
1138
1139 If ShipToFnd = 'N';
1140     H1ShTo = BillToNum;
1141
1142     // Figure the Drop Ship Number for this Order
1143     Exec SQL
1144     Select DSNMBR Into :DropShip
1145     From HDDSHP
1146     Where DSVCF = 'C' And DSVNCS = :HDPCust#
1147         And DSNAME = :STName1 And DSADR1 = :STADR1
1148         And DSADR2 = :STADR2 And DSADR3 = :STADR3
1149         And DSCITY = :STCITY And DSST = :STSTATE
1150         And DSZIP = :STZIP And DSCTRY = :STCOUNTRY
1151         And DSCNTC = :CMCNTC And DSC TTC = :CMCTTC
1152         And DSLOC1 = :CMLOC1 And DSLOC2 = :CMLOC2
1153         And DSLOC3 = :CMLOC3;
1154
1155 If SQLCod <> 0;
1156
1157     Exec SQL
1158     Select Max(DSNMBR) Into :DropShip from HDDSHP;
1159
1160     DropShip += 1;
1161
1162     Exec SQL
1163     Insert Into HDDSHP
1164     (DSVCF, DSVNCS, DSNMBR, DSNAME, DSADR1, DSADR2, DSADR3,
1165     DSCITY, DSST, DSZIP, DSCTRY, DSCNTC, DSC TTC, DSLOC1,
1166     DSLOC2, DSLOC3, DSDATE, DSPURG, DSNAMEU, DSADR1U,
1167     DSADR2U, DSADR3U, DSCITYU)
1168     Values('C', :HDPCust#, :DropShip, :STName1, :STAdr1,
1169     :STAdr2, :STAdr3, :STCity, :STState, :STZip,
1170     :STCOUNTRY, :CMCNTC, :CMCTTC, :CMLOC1, :CMLOC2,
1171     :CMLOC3, :H1RQDT, 'Y', UCASE(:STName1),
1172     UCASE(:STADR1), UCASE(:STAdr2), UCASE(:STAdr3),
1173     UCASE(:STCity));
1174
1175     H1DSHP = DropShip;
1176
1177     // *** Notify bbusch@screen-graphics.com of ***
1178     // *** the new customer that needs to get ***
1179     // *** set up using the drop ship info ***
1180     Alert = 'XMLDrpShp';
1181     If xmlMode = 'T';
1182         AlertSubj = '*** TEST ***';
1183     EndIf;
1184     AlertSubj = %Trim(AlertSubj) + ' ' +
1185     'Ariba Order Alert: Drop Ship Added';
1186     AlertMsg = 'Ariba Customer: ' + xmlRetailer + CrLf;
1187     AlertMsg = %Trim(AlertMsg) + 'HarrisData Order: '

```

```

1188         + %Char(LastOrder)
1189         + CrLf;
1190     AlertMsg = %Trim(AlertMsg) + 'Ariba Order: '
1191         + HWPO# + CrLf;
1192     AlertMsg = %Trim(AlertMsg) + 'New Drop Ship: '
1193         + %Char(DropShip) + CrLf + CrLf;
1194     AlertMsg = %Trim(AlertMsg) + 'Code: '
1195         + HWShpToID + CrLf;
1196     AlertMsg = %Trim(AlertMsg) + 'Address: '
1197         + %Trim(STName1) + CrLf;
1198     AlertMsg = %Trim(AlertMsg) + '          ' + %Trim(STAdr1) +
1199         CrLf;
1200     AlertMsg = %Trim(AlertMsg) + '          ' + %Trim(STAdr2) +
1201         CrLf;
1202     AlertMsg = %Trim(AlertMsg) + '          ' + %Trim(STAdr3) +
1203         CrLf;
1204     AlertMsg = %Trim(AlertMsg) + '          ' + %Trim(STCity) +
1205         ', ' + %Trim(STState) + ' ' + %Trim(STZip) +
1206         CrLf + CrLf;
1207     AlertMsg = %Trim(AlertMsg) + 'Issue: CMUDF4 info right?'
1208         + CrLf;
1209     Exsr Notify;
1210 Else;
1211
1212     H1DSHP = DropShip;
1213
1214     EndIf;
1215
1216 EndIf;
1217
1218
1219
1220     CustomItem = 'N';
1221
1222     // Write the detail for the order...
1223 +   C13     // C13 - count the detail lines actually written
1224 +   C13     ProcessedItems = 0;
1225 +   C13
1226     For lineCounter = 1 to 500;
1227
1228         Clear OEDTWKR;
1229         Clear XMLOEDTR;
1230
1231         // If the line is not there it will throw an error, exit
1232         // at that point because all items have been processed.
1233
1234         Monitor;
1235
1236         BuyerOrdLn = cXML.Request.
1237             OrderRequest(orderCounter).
1238             ItemOut(lineCounter).
1239             lineNumber;
1240
1241         // If BuyerOrdLn > 999;
1242             01OrL# = lineCounter;

```

```

1243 //      Else;
1244 //      01OrL# = BuyerOrdLn;
1245 //      EndIf;
1246
1247 On-Error;
1248 Leave;
1249 EndMon;
1250
1251 01VItem = cXML.Request.
1252         OrderRequest(orderCounter).
1253         ItemOut(lineCounter).ItemID.
1254         SupplierPartID;
1255 01VItem = %XLate(Lo:Up:01VItem);
1256
1257 // Exit the loop if the last line was just processed...
1258 If 01VItem = *Blanks;
1259     Leave;
1260 EndIf;
1261
1262 01ITEM = 01VITEM;
1263
1264 01BItem = cXML.Request.
1265         OrderRequest(orderCounter).
1266         ItemOut(lineCounter).ItemID.
1267         SupplierPartID;
1268 01BItem = %XLate(Lo:Up:01BItem);
1269
1270 Monitor;
1271 01QOrd = %Dec(%XLATE('$*',' ':cXML.Request.
1272         OrderRequest(orderCounter).
1273         ItemOut(lineCounter).
1274         quantity) : 13 : 4);
1275 // On-Error 105;
1276 On-Error;
1277 01QOrd = 1;
1278 EndMon;
1279
1280 01QUOM = 'EA';
1281
1282 // ***** //
1283 // If the item is NOT found on HDIMST it is to be processed as a //
1284 // custom, non-stock item ...per Bill Busch 10/15/2019 //
1285 // ***** //
1286
1287 // Description is longer than HD's OEORDT ODIMDS field, so
1288 // check to see if the description includes IMIMDS and, if
1289 // so write the remainder of the SAVEDESC after the ";" to
1290 // OECMWK.
1291 Chain 01ITEM HDIMST;
1292
1293 // Item is not on HDIMST so set it as a 'CUSTOM ITEM'
1294 If NOT %Found(HDIMST);
1295     CustomItem = 'Y';
1296 + C7 // C7 - a failed chain leaves the prior line's
1297 + C7 // IMPCLS in the record. Never read IMPCLS

```



```

1298 + C7 // directly again; use WkClass.
1299 + C7 WkClass = *Blanks;
1300 O1OREC = 'N';
1301 O1ITEM = 'CUSTOM ITEM';
1302 O1PCLS = '9MIS';
1303 O1STUM = 'EA';
1304 O1PRUM = 'EA';
1305 O1ORUF = 1.0000;
1306 O1TAXC = 'Y';
1307 Else;
1308 CustomItem = 'N';
1309 + C7 WkClass = IMPCLS; // C7
1310 O1IMDS = IMIMDS;
1311 O1OREC = 'S';
1312 O1PCLS = IMPCLS;
1313 O1STUM = IMUOMS;
1314 EndIf;
1315
1316 SaveDesc = cXML.Request.OrderRequest(orderCounter).
1317 ItemOut(lineCounter).ItemDetail.Description;
1318
1319 SaveDesc = %Subst(SaveDesc:3:298);
1320
1321 // Search/Replace any invalid XML characters in the description
1322
1323 // apostrophe...
1324 CharPos = %Scan(apos: SaveDesc);
1325 DoW CharPos > 0;
1326 SaveDesc = %Replace('': SaveDesc: CharPos:
1327 %len(apos));
1328 CharPos = %Scan(apos: SaveDesc: CharPos + 1);
1329 EndDo;
1330
1331 // quote...
1332 CharPos = %Scan(quot: SaveDesc);
1333 DoW CharPos > 0;
1334 SaveDesc = %Replace('': SaveDesc: CharPos:
1335 %len(quot));
1336 CharPos = %Scan(quot: SaveDesc: CharPos + 1);
1337 EndDo;
1338
1339 // ampersand...
1340 CharPos = %Scan(amper: SaveDesc);
1341 DoW CharPos > 0;
1342 SaveDesc = %Replace('&': SaveDesc: CharPos:
1343 %len(amper));
1344 CharPos = %Scan(amper: SaveDesc: CharPos + 1);
1345 EndDo;
1346
1347 // less than...
1348 CharPos = %Scan(lt: SaveDesc);
1349 DoW CharPos > 0;
1350 SaveDesc = %Replace('<': SaveDesc: CharPos:
1351 %len(lt));
1352 CharPos = %Scan(lt: SaveDesc: CharPos + 1);

```

```

1353      EndDo;
1354
1355      // greater than...
1356      CharPos = %Scan(gt: SaveDesc);
1357      DoW CharPos > 0;
1358          SaveDesc = %Replace('>': SaveDesc: CharPos:
1359                          %len(gt));
1360          CharPos = %Scan(gt: SaveDesc: CharPos + 1);
1361      EndDo;
1362
1363
1364      Clear OECMWKR;
1365
1366      If CustomItem = 'N';
1367          CharPos = %Scan('; ':SaveDesc);
1368          If CharPos > 0;
1369              SaveDesc = %Subst(SaveDesc : CharPos + 1 :
1370                              %Len(%Trim(SaveDesc)) - CharPos + 1);
1371              Exsr WriteCmt;
1372          EndIf;
1373      Else;
1374          Exsr WriteCmt;
1375      EndIf;
1376
1377      01SlPrXlt = %XLATE('$*',' ':cXML.Request.
1378                      OrderRequest(orderCounter).
1379                      ItemOut(lineCounter).ItemDetail.
1380                      UnitPrice.Money.dsvalue);
1381

```

```

- C17 ----- 01SlPr = %Dec(01SlPrXlt:13:5);

```

```

1382 + C17      // C17 - a malformed price used to halt the program.
1383 + C17      //      The quantity conversion above is monitored;
1384 + C17      //      this one now matches it.
1385 + C17      Monitor;
1386 + C17          01SlPr = %Dec(01SlPrXlt:13:5);
1387 + C17      On-Error;
1388 + C17          01SlPr = 0;
1389 + C17      EndMon;

```

```

1390
1391      01CUOM = 'EA';
1392
1393
1394      // Populate the OEDTWK fields...
1395      01OCTL = LastOrder;
1396      01ORL# = lineCounter;
1397
1398      01MNCD = 'A';
1399      01BLKT = 'N';
1400      01PLT = 5;
1401      01TAXC = 'N';
1402      01LPOV = 'Y';
1403      01CPOV = 'Y';
1404      01STKL = 'Y';
1405      01LLTC = 'N';
1406      If H1DSHP = 0;

```

```

1407         01DSHP = 'N';
1408     Else;
1409         01DSHP = 'Y';
1410     EndIf;
1411     01GNPO = 'N';
1412
1413     // Get the default warehouse for this trading partner...
1414     Chain HDCust# HDCUSTR;
1415 +   C15
1416 +   C15     // C15 - a failed chain must not leak the prior
1417 +   C15     //      customer's ship via, warehouse or location.
1418 +   C15     If NOT %Found(HDCUST);
1419 +   C15         Clear HDCUSTR;
1420 +   C15     EndIf;
1421 +   C15
1422 +   C15     // C9 - customer class drives the prefix lead-day
1423 +   C15     //      lookup. No extra I/O, the chain is already here.
1424 +   C15     CustClass = CMCCLS;
1425
1426     If ShipToFnd = 'Y';
1427         01SVSV = CMSVSV;
1428     Else;
1429         01SVSV = CMSV;
1430     EndIf;
1431
1432     // Populate 01SVDS on OEDTWK for HOEPIC pick ticket Ship Via
1433     Chain 01SVSV HDSHPV;
1434     If %Found(HDSHPV);
1435         01SVDS = SVSVDS;
1436     Else;
1437         01SVDS = *Blanks;
1438     EndIf;
1439
1440     01WH   = CMWH#;
1441     01BOAL = *Blanks;
1442     01OCTL = LastOrder;
1443     01LOC# = CMLOC#;
1444     01SHTO = H1SHTO;
1445     01ORST = 'O';
1446
1447
1448     Chain (01ITEM:01WH) HDIWHS;
1449     If %Found(HDIWHS);
1450         01PGRP = IWPGRP;
1451         01PRUM = IWUOMP;
1452         01ORUF = IWPCPP;
1453         01TAXC = IWTXCD;
1454         // Per Bill Busch 10/15/2019 ... ALWAYS USE PLANT 1 for anything
1455         // coming in via XML from Ariba
1456         // 01PLT = IWPLT;
1457         01PLT = 1;
1458     Else;
1459 +   C8     // C8 - a failed chain must not leak the prior
1460 +   C8     //      line's quantities into Avail.
1461 +   C8     IWOHQT = 0;

```

```

1462 + C8 IWQ00 = 0;
1463 + C8 IWRESQ = 0;
1464 O1PGRP = *Blanks;
1465 O1PRUM = 'EA';
1466 O1PLT = 1;
1467 O1ORUF = 1.0000;
1468 O1TAXC = STTxCd;
1469 EndIf;
1470
1471 Chain (O1PLT:O1ITEM) HDIPLT;
1472
1473 If %Found(HDIPLT);
1474
1475 Avail = (IWOHQT + IWQ00) - (IWRESQ + IPCMTO);
1476 + C4 LeadDays = IPTLT; // C4
1477
1478 Else;
1479 + C4 LeadDays = 0; // C4
1480
1481 Alert = 'XMLConfig';
1482 If xmlMode = 'T';
1483 AlertSubj = '*** TEST ***';
1484 EndIf;
1485 AlertSubj = %Trim(AlertSubj) + ' ' +
1486 'Coupa Order Alert: STOCK ITEM NO SET UP';
1487 AlertMsg = 'Coupa Customer: ' + xmlRetailer + CrLf;
1488 AlertMsg = %Trim(AlertMsg) + 'HarrisData Order: '
1489 + %Char(LastOrder) + CrLf;
1490 AlertMsg = %Trim(AlertMsg) + 'Coupa Order: '
1491 + HWPO# + CrLf;
1492 AlertMsg = %Trim(AlertMsg) + 'Order Line: '
1493 + %Char(O1OrL#) + CrLf;
1494 AlertMsg = %Trim(AlertMsg) + 'Item: '
1495 + O1BItem + CrLf;
1496 AlertMsg = %Trim(AlertMsg) + 'Issue: Configuration'
1497 + CrLf;
1498 AlertMsg = %Trim(AlertMsg) +
1499 'Details: Stock item was not found ' +
1500 'on the HarrisData Item Plant file.';
1501
1502 Exsr Notify;
1503
1504 Avail = 0;
1505
1506 EndIf;

```

```

- C4 -
- C4 -
- C4 ----- // Determine the number of days to add to the required date --
- C4 ----- // given Bill's criteria defined here+
- C4 -
- C4 ----- // Set the required date by item based on the criteria Bill provided 6/13/2023,
- C4 ----- // as follows:
- C4 ----- //
- C4 ----- // 1) Stock will equal any item that begins with '9' and has a 3rd
- C4 ----- // character that's ' ' OR begins with '3M0'.

```

```

- C4 ----- // These items will be checked for availability and customer service will be
- C4 ----- // notified of shorts. (hdiwhs.IWQHQT hdiwhs.IWQOPK + hdiwhs.IWRESQ +
- C4 ----- // hdipl1.IPCMT0) ...is the calc as provided by Bill.
- C4 ----- //
- C4 ----- // 2) Stock NOT in stock will be get 5 business days added to its required
- C4 ----- // date.
- C4 ----- //
- C4 ----- // 3) All digital cuts (product classes 'DCCU' and 'DCSG')
- C4 ----- // will have 5 business days added to its required date.
- C4 ----- //
- C4 ----- // 4) Kits (product class 'KITS') will have 7 days added
- C4 ----- // to it.
- C4 ----- //
- C4 ----- // 6) Custom Items is everything else and they will have
- C4 ----- // 12 days added to the order date.
- C4 -
- C4 -
- C4 ----- // Clear AddDays for each item so CalcReqD is accurate per each item.
- C4 ----- AddDays = 0;
- C4 -
- C4 ----- //
- C4 ----- // Availability FIRST, then the UDC override (per Bill).
- C4 ----- // If available -> honor the per item UDC override (or the
- C4 ----- // in stock default of 1 day). If short -> fall through to the
- C4 ----- // generic day count rules below.
- C4 ----- //
- C4 -
- C4 ----- If Avail -> 01QOrd;
- C4 -
- C4 ----- // Available -> honor the per item UDC override
- C4 ----- Exec SQL
- C4 ----- Select Coalesce(Cast(UDCDESC1 As Decimal(2,0)), 0)
- C4 ----- Into :UDCDays :NullInd
- C4 ----- from ProITRG/UDCDETAIL
- C4 ----- Where UDCSYSTEMD = 'ORDERLINES' And UDCCODED = 'REQUIREDDAY'
- C4 ----- And UDCKEY = :01BItem;
- C4 -
- C4 ----- If UDCDays > 0; ----- // on the UDC table
- C4 ----- AddDays = UDCDays;
- C4 ----- Else; ----- // not on the table -> in stock default
- C4 ----- AddDays = 1;
- C4 ----- EndIf;
- C4 -
- C4 ----- Else;
- C4 -
- C4 ----- // Not available -> generic day count rules
- C4 ----- Select;
- C4 -
- C4 ----- When impels = 'DCCU' Or
- C4 ----- impels = 'DCSG';
- C4 -
- C4 ----- // KIT (Digital Cut) product class DCCU / DCSG -> 5 days
- C4 ----- AddDays = 5;
- C4 -
- C4 ----- When impels = 'KITS';

```

```

- C4 -
- C4 - // KIT (Digital / Screen-Print) - product class KITS -> 10 days
- C4 - AddDays = 10;
- C4 -
- C4 - When (%Subst(01BItem:1:1) = '9' And
- C4 - %Subst(01BItem:3:1) = ' ') Or
- C4 - %Subst(01BItem:1:3) = '3M0';
- C4 -
- C4 - // Stock item, NOT in stock -> 5 business days and notify CS
- C4 - StockItem = 'V';
- C4 - AddDays = 5;
- C4 -
- C4 - Alert = 'XMLAvail';
- C4 - If xmlMode = 'T';
- C4 - AlertSubj = '*** TEST ***';
- C4 - EndIf;
- C4 - AlertSubj = %Trim(AlertSubj) + ' ' +
- C4 - 'Ariba Order Alert: STOCK ITEM AVAILABILITY';
- C4 - AlertMsg = 'Ariba Customer: ' + xmlRetailer + CrLf;
- C4 - AlertMsg = %Trim(AlertMsg) + 'HarrisData Order: '
- C4 - + %Char(LastOrder) + CrLf;
- C4 - AlertMsg = %Trim(AlertMsg) + 'OSN Order: '
- C4 - + HWPO# + CrLf;
- C4 - AlertMsg = %Trim(AlertMsg) + 'Order Line: '
- C4 - + %Char(01OrL#) + CrLf;
- C4 - AlertMsg = %Trim(AlertMsg) + 'Item: '
- C4 - + 01BItem + CrLf;
- C4 - AlertMsg = %Trim(AlertMsg) + 'Issue: Availability'
- C4 - + CrLf;
- C4 - AlertMsg = %Trim(AlertMsg) + 'Details: ' +
- C4 - %Char(01QOrd) +
- C4 - ' Ordered Only ' + %Char(Avail) +
- C4 - ' are available.';
- C4 - Exsr Notify;
- C4 -
- C4 - Other;
- C4 -
- C4 - // Custom / non stock item -> 10 days
- C4 - AddDays = 10;
- C4 -
- C4 - EndS1;
- C4 -
- C4 - EndIf;
- C4 -
- C4 -
- C4 - //
- C4 - // Changed per Bill on 05/17/2023... moved calculation for
- C4 - // the required date down to after availability check.
- C4 - //
- C4 -

```

```

1507 + C4 //=====
1508 + C4 // C1/C2 Hardcoded product-class branches (DCCU / DCSG / KITS)
1509 + C4 // and the 9x- / 3M0 stock-name test are removed. The
1510 + C4 // XMLAvail short-stock alert lived inside that block and

```

```

1511 + C4 // is removed with it.
1512 + C4 // C4 The required-date day count now comes entirely from the
1513 + C4 // UDC file, in this order:
1514 + C4 // 1. ORDERLINES / REQUIREDAY on the item
1515 + C4 // 2. ORDERLINES / PREFIXRQDD on the item prefix,
1516 + C4 // scoped to this customer's class, or the
1517 + C4 // all-customers key 'SCREEN GRAPHICS'
1518 + C4 // 3. ORDERLINES / PCREQUIRED on the product class
1519 + C4 // 4. HDIPLT.IPTLT, only when above 1 day
1520 + C4 // 5. default of 10
1521 + C4 // In stock always returns 1 business day.
1522 + C4 // CalcReqD skips weekends and HOLIDAY rows, so every
1523 + C4 // value set below is a business-day count.
1524 + C4 //=====
1525 + C4
1526 + C4 AddDays = 0;
1527 + C4
1528 + C4 If Avail >= 01Q0rd;
1529 + C4
1530 + C4 // In stock -> next business day. No lookup: an item with
1531 + C4 // exact or more available is always 1 day.
1532 + C4 AddDays = 1;
1533 + C4
1534 + C4 Else;
1535 + C4
1536 + C4 // ReqDays is zeroed first and each tier runs only while it is
1537 + C4 // still zero, so a no-row condition (SQLCODE 100) can never
1538 + C4 // carry a prior line's value forward.
1539 + C4 ReqDays = 0;
1540 + C4
1541 + C4 // 1. exact item
1542 + C4 Exec SQL
1543 + C4 Select Coalesce(Cast(UDCDESC1 As Decimal(3,0)),0)
1544 + C4 Into :ReqDays
1545 + C4 From ProITRG/UDCDETAIL
1546 + C4 Where UDCSYSTEMD = 'ORDERLINES'
1547 + C4 And UDCCODED = 'REQUIREDAY'
1548 + C4 And UDCKEY = :01BItem;
1549 + C4
1550 + C4 // 2. item prefix, scoped by customer class. UDCKEY holds
1551 + C4 // either a 2 character HDCCLS class code, which must match
1552 + C4 // this customer's CMCCLS, or the literal 'SCREEN GRAPHICS'
1553 + C4 // which applies to every customer. A customer specific row
1554 + C4 // outranks 'SCREEN GRAPHICS'; among equals the longest
1555 + C4 // prefix wins.
1556 + C4 If ReqDays = 0;
1557 + C4 Exec SQL
1558 + C4 Select Coalesce(Cast(UDCDESC1 As Decimal(3,0)),0)
1559 + C4 Into :ReqDays
1560 + C4 From ProITRG/UDCDETAIL
1561 + C4 Where UDCSYSTEMD = 'ORDERLINES'
1562 + C4 And UDCCODED = 'PREFIXRQDD'
1563 + C4 And :01BItem Like Concat(RTrim(UDCINDEXED),'%')
1564 + C4 And (UDCKEY = :CustClass Or
1565 + C4 UDCKEY = 'SCREEN GRAPHICS')

```

```

1566 + C4          Order By Case When UDCKEY = 'SCREEN GRAPHICS'
1567 + C4              Then 1 Else 0 End,
1568 + C4              Length(RTrim(UDCINDEXED)) Desc
1569 + C4          Fetch First 1 Row Only;
1570 + C4      EndIf;
1571 + C4
1572 + C4      // 3. product class, taken from the guarded HDIMST chain
1573 + C4      If ReqDays = 0;
1574 + C4          Exec SQL
1575 + C4              Select Coalesce(Cast(UDCDESC1 As Decimal(3,0)),0)
1576 + C4                  Into :ReqDays
1577 + C4                  From ProITRG/UDCDETAIL
1578 + C4                  Where UDCSYSTEMD = 'ORDERLINES'
1579 + C4                      And UDCCODED  = 'PCREQUIRED'
1580 + C4                      And UDCKEY    = :WkClass;
1581 + C4      EndIf;
1582 + C4
1583 + C4      Select;
1584 + C4      When ReqDays > 0;
1585 + C4          AddDays = ReqDays;
1586 + C4
1587 + C4      When LeadDays > 1;
1588 + C4          // Plant lead time. IPTLT is DECIMAL(5,1), so round a partial
1589 + C4          // day up: a line cannot ship in part of a business day.
1590 + C4          // A value of 1 or less is a ship-from-stock figure and is
1591 + C4          // meaningless for a short line, so it falls through to the
1592 + C4          // default rather than promising next day on a short order.
1593 + C4          AddDays = %Int(LeadDays);
1594 + C4          If LeadDays > AddDays;
1595 + C4              AddDays += 1;
1596 + C4          EndIf;
1597 + C4
1598 + C4      Other;
1599 + C4          AddDays = 10;
1600 + C4
1601 + C4      EndSl;
1602 + C4
1603 + C4      EndIf;
1604      Exsr CalcReqD;
1605
1606      // -----
1607
1608      Chain (01Plt:1:01ITEM) HDMCMM;
1609      If %Found(HDMCMM);
1610          01UCC1 = CMUCC1;
1611          01UCC2 = CMUCC2;
1612          01UCC3 = CMUCC3;
1613          01UCC4 = CMUCC4;
1614          01UCC5 = CMUCC5;
1615          01COST = 01UCC1 + 01UCC2 + 01UCC3 + 01UCC4 + 01UCC5;
1616      EndIf;
1617
1618      01LIST = 01QORD * 01SlPr;
1619      OrdTotal += 01List;
1620      01QAFG = '1';

```



```

1621         01UDA1 = %Subst(giftMSG:1:15);
1622         01UDA1 = %XLate(Lo:Up:01UDA1);
1623         01UDA2 = %Subst(giftMSG:16:15);
1624         01UDA2 = %XLate(Lo:Up:01UDA2);
1625         01UDA3 = %Subst(giftMSG:31:15);
1626         01UDA3 = %XLate(Lo:Up:01UDA3);
1627         01UDA4 = %Subst(giftMSG:45);
1628         01UDA4 = %XLate(Lo:Up:01UDA4);
1629
1630         // 01RQDT (Detail Required Date) is calculated in CalcReqD.
1631         // It was removed from here because it was being overwritten
1632         // by LastDtIDate which was incorrect here whereas 01RQDT
1633         // from the subroutine was correct and should've been used.
1634
1635         Write OEDTWKR;
1636 +      C13      ProcessedItems += 1;                      // C13
1637
1638
1639         XDOBAT = HWPO#;
1640         XDSNDR = cXML.Header.From.Credential.Identity;
1641         // XDXORD = %Dec(XMLOrderID:11:0);
1642         XDXORD = 0;
1643         XDPO = HWPO#;
1644         XDHDR = LastOrder;
1645         XDOLIN = lineCounter;
1646         XDLNIID = %Char(lineCounter);
1647         XDMLin = %Char(cXML.Request.OrderRequest(orderCounter).
1648                     ItemOut(lineCounter).lineNumber) ;
1649
1650         // XDQty0 = %Dec(%XLATE('$*',':':'':cXML.Request.
1651         //           OrderRequest(orderCounter).
1652         //           ItemOut(lineCounter).
1653         //           quantity) : 13 : 4);
1654         XDQty0 = 01Q0rd ;
1655
1656         XDUOM = 'EA';
1657         XDDESC = 01IMDS;
1658         XDMSKU = cXML.Request.OrderRequest(orderCounter).
1659                 ItemOut(lineCounter).ItemID.SupplierPartID;
1660
1661         XDUCST = 01S1Pr;
1662         XDLSHP = 01SVSv;
1663         XDLSHP = %XLate(Lo:Up:XDLSHP);
1664         XDEShp = %char(LastDtIDate:*ymd0);
1665
1666         Write XMLOEDTR;
1667
1668     EndFor;
1669 +      C13
1670 +      C13      //=====
1671 +      C13      // C13 Order arrived with no detail lines. Alert only. The order
1672 +      C13      // IS still created: execution falls straight through to the
1673 +      C13      // H1RqDt fallback, Write OEHDWKR, the tax call, the data queue
1674 +      C13      // and the XMLOEHD write, exactly as it does today.
1675 +      C13      // Do NOT add LeaveSR here.

```

```

1676 + C13 //=====
1677 + C13 If ProcessedItems = 0;
1678 + C13
1679 + C13     Alert      = 'XMLNODTL';
1680 + C13     AlertSubj = *Blanks;
1681 + C13     If xmlMode = 'T';
1682 + C13         AlertSubj = '*** TEST ***';
1683 + C13     EndIf;
1684 + C13     AlertSubj = %Trim(AlertSubj) + ' ' +
1685 + C13         'Imported Order Alert: Order has no detail';
1686 + C13     AlertMsg = 'Customer: ' + %Trim(xmlRetailer) + CrLf;
1687 + C13     AlertMsg = %Trim(AlertMsg) + 'Imported File: ' +
1688 + C13         %Trim(ifsFile) + CrLf;
1689 + C13     AlertMsg = %Trim(AlertMsg) + 'Imported Order: ' +
1690 + C13         %Trim(HWPO#) + CrLf;
1691 + C13     AlertMsg = %Trim(AlertMsg) + 'Issue: No Items Ordered' + CrLf;
1692 + C13     Exsr Notify;
1693 + C13
1694 + C13 EndIf;
1695 + C13 //=====

```

```

1696
1697 // No line set a date -> fall back to the next business day
1698 If H1RqDt = 0;
1699     H1RqDt = %Dec(%Char(LastDt1Date:*CYMD0):7:0);
1700 EndIf;
1701
1702 // Write the OEHDWK record with the order total...
1703
1704 H1OCTL = LastOrder;
1705 H1ORD# = LastOrder;
1706 H1MNCD = 'A';
1707 H1BLTO = HDPCust#;
1708
1709 Chain H1BLTO HDCUSTR;

```

```

1710 + C16
1711 + C16 // C16 - a failed chain must not leave the previous
1712 + C16 //      customer's values on the order header.
1713 + C16 If NOT %Found(HDCUST);
1714 + C16     Clear HDCUSTR;
1715 + C16 EndIf;

```

```

1716
1717 H1Bank = 7;
1718 H1Loc# = CMLoc#;
1719
1720 H1SLSM = CMSLSM;
1721
1722 Clear HDTAXDR;
1723 Chain (STState: 0: 0: 0: 0: 0) HDTAXDR;
1724
1725 H1STTX = TDSTCD;
1726 If ShipToFnd <> 'Y';
1727     H1CTXC = CMCTXC;
1728 Else;
1729     H1CTXC = STTxCd;
1730 EndIf;

```

```

1731      H1CNTX = CMCNTC;
1732      H1CTTX = CMCTTC;
1733      H1LOC1 = CMLOC1;
1734      H1LOC2 = CMLOC2;
1735      H1LOC3 = CMLOC3;
1736      H1BDTE = %dec(%char(%date(%Subst(HWOrdDate:1:10):*iso-):*cymd0):7:0);
1737      H1ORTY = CMORTY;
1738
1739      H1ORRF = HWPO#;
1740      H1CTRM = CMCTRM;
1741      If ShipToFnd = 'Y';
1742          H1SVSV = CMSVSV;
1743      Else;
1744          H1SVSV = CMSV;
1745      EndIf;
1746
1747      // Pull the Ship Via Description
1748      Chain H1SVSV HDSHPV;
1749      If %Found(HDSHPV);
1750          H1SVDS = SVSVDS;
1751      Else;
1752          H1SVDS = *Blanks;
1753      EndIf;
1754
1755      H1BOAL = *Blanks;
1756      H1TOT = OrdTotal;
1757      H1FBCD = 'OR';
1758
1759      // Set Order Flags (Pulled directly from HOEOEM)
1760
1761      FGORTY = CMORTY;
1762
1763      /End-Free
1764      C*
1765      C      KFG          KLIST
1766      C          KFLD          APID
1767      C          KFLD          FGORTY
1768      C*
1769      C          CLEAR          DFL
1770      C      KFG          SETLL      HDORFGR
1771      C          DOU          1=2
1772      C      KFG          READE      HDORFGR
1773      C          IF          %EOF(HDORFG)
1774      C          LEAVE
1775      C          ENDIF
1776      C          Z-ADD      FGFGFG      F
1777      C          MOVE      FGFGOP      DFL(F)
1778      C          ENDDO
1779      C*
1780      C          MOVEA      DFL          H10EF
1781      C*
1782      /Free
1783
1784      // H1RQDT (Header Required Date) is derived initially from the
1785      // NxtHDOrd routine as the next business day from when the order

```

```

1786 // came in. It can be overwritten in CalcReqD because Bill now
1787 // wants the earliest ship date for a line item to be the order
1788 // header required date... NOT the latest item's ship date...
1789 // per a phone conversation while debugging on 6/13/2023.
1790
1791 Write OEHDWKR;
1792
1793
1794
1795 // Calculate taxes and add them to the order...
1796 xmlOCTL = LastOrder;
1797 xmlFRT = 0.00;
1798 xmlCHRG = 0.00;
1799 xmlUPDTOT = 'R';
1800 HOEUMC_W(HexHnd : xmlDBID : xmlOCTL : xmlFRT : xmlCHRG : xmlUPDTOT);
1801
1802
1803 // Notify customerservice@screen-graphics.com if this order had any
1804 // "CUSTOM ITEMS" on it. Specify all the normal detail information
1805 // but specify each "CUSTOM ITEM" detail line by number with its
1806 // full description, quantity, price, and extended price.
1807
1808 If CustomItem = 'Y';
1809     Exsr AlertCustom;
1810 EndIf;
1811
1812
1813 // Submit the order to the Data Queue...
1814 OQTTP = 'ORVW';
1815 OQTURN = 0;
1816 OQORD# = LastOrder;
1817 OQOCTL = LastOrder;
1818 OQTSTP = %TimeStamp();
1819 OQTSUS = %SUBST(XMLRetailer:1:10);
1820 OQTSWS = 'XML850R_AR';
1821 OQPGM = 'HOEOEM';
1822 QSNDDTAQ(DQ#NAM:DQ#LIB:DQ#LEN:DTAQ);
1823
1824
1825 // Now write the order to the XML Orders files...
1826 Clear XMLOEHDR;
1827
1828 XHFILE = XMLFILE;
1829 XHOBAT = XMLOrderID;
1830 XHPTNR = XMLRetailer;
1831 XHSNDR = 'Amerigas';
1832 XHTRNS = Transact;
1833 // XHXORD = %Dec(XMLOrderID:11:0);
1834 XHXORD = 0;
1835 XHPO = HWPO#;
1836 XHHDR = LastOrder;
1837 XHOLNT = HWLinCnt;
1838 XHPOTY = HWPOType;
1839 XHGFTM = HWGiftMSG;
1840 XHODTE = HWOrdDate;

```

```

1841      XHSHCD = HWShipCd;
1842      XHCNTL = HWControl;
1843      XHSLDV = HWSlsDiv;
1844      XHCORD = XMLOrderID;
1845      XHBCNT = HWBuyCntrct;
1846      XHMDPT = HWMerchDpt;
1847      XHRQSD = %EditC(H1RqDt:'X');
1848      // Capture Ship-To Address Only...
1849      XSHSPP      = HWShpToID;
1850      XHPPNM1     = STName1;
1851      XHPP1A1     = STAdr1;
1852      XHPP1A2     = STAdr2;
1853      XHCTY1     = STCity;
1854      XHST1      = STState;
1855      XHCNT1     = STCountry;
1856      XHZIP1     = STZIP;
1857      XHEML1     = STEmail;
1858      XHDPH1     = STPhone;
1859      XHBLPP     = HWBillToID;
1860      XHCUPP     = HWCustID;
1861      XHIVPP     = HWInvToID;
1862
1863      Write XMLOEHDR;
1864
1865
1866      EndSr;
1867      //-----*
1868
1869
1870
1871      //-----*
1872      // WriteCmt will handle writing detail line item comments to an *
1873      // order. *
1874      //-----*
1875
1876      BegSr WriteCmt;
1877
1878      For x = 1 to 5;
1879          04CSEQ = x;
1880          Select;
1881              When x = 1;
1882                  04CMNT = %Subst(SaveDesc : 1 : 60);
1883              When x = 2;
1884                  04CMNT = %Subst(SaveDesc : 61 : 60);
1885              When x = 3;
1886                  04CMNT = %Subst(SaveDesc : 121 : 60);
1887              When x = 4;
1888                  04CMNT = %Subst(SaveDesc : 181 : 60);
1889              When x = 5;
1890                  04CMNT = %Subst(SaveDesc : 241 : 60);
1891          EndSl;
1892
1893      If 04CMNT <> *Blanks;
1894          04OCTL = LastOrder;
1895          04ORL# = lineCounter;

```

```

1896      04BLN# = 0;
1897      04DOCT = 'ACK';
1898      04CSR = *BLanks;
1899
1900      Chain (04OCTL:04ORL#:04BLN#:04DOCT:04CSEQ) OECMWKR;
1901      If NOT %Found(OECMWK);
1902          Write OECMWKR;
1903      EndIf;
1904      EndIf;
1905      EndFor;
1906
1907      EndSr;
1908      //-----*
1909
1910
1911
1912
1913      //-----*
1914      /* AlertCustom notifies customerservice@screen-grpahics.com of *
1915      /* the orders with 'CUSTOM ITEM's on them and the details about *
1916      /* those lines. *
1917      //-----*
1918      BegSr AlertCustom;
1919
1920      If xmlMode = 'T';
1921          AlertSubj = '*** TEST ***' ;
1922      EndIf;
1923
1924      AlertSubj = %Trim(AlertSubj) + ' ' +
1925          'OSN Order Alert: CUSTOM ITEMs ordered';
1926      AlertMsg = 'OSN Customer: ' + xmlRetailer + CrLf;
1927      AlertMsg = %Trim(AlertMsg) + 'HarrisData Order: '
1928          + %Char(LastOrder) + CrLf;
1929      AlertMsg = %Trim(AlertMsg) + 'Republic OSN order: '
1930          + HWPO# + CrLf;
1931
1932      // Now pull the order from OEDTWK, each 'CUSTOM ITEM' line,
1933      // and report all the details about those lines for customer service.
1934      Exec SQL
1935          Declare Custom Cursor for
1936          Select O1ORL#, O1RQDT, O1IMDS, O4CMNT, O1QORD, O1SLPR
1937          from OEDTWK Left Outer Join OECMWK on
1938              O1OCTL = O4OCTL And O1ORL# = O4ORL# And
1939              O4DOCT = 'INT' And O4CSEQ = 1
1940          Where O1ITEM = 'CUSTOM ITEM' and
1941              O1OCTL = :LastOrder ;
1942
1943      Exec SQL
1944          Open Custom;
1945
1946      Exec SQL
1947          Fetch Next from Custom Into :CustOrdLn, :CustRqDt,
1948              :CustDesc, :CustCmt,
1949              :CustQOrd, :CustSlPr;
1950

```

```

1951      pass = 0;
1952
1953      DoW SQLCod = 0;
1954
1955      pass += 1;
1956
1957      If pass > 1;
1958          AlertMsg = %Trim(AlertMsg) + CrLf;
1959      EndIf;
1960
1961      AlertMsg = %Trim(AlertMsg) + 'Order Line: '
1962                + %Char(CustOrdLn) + CrLf;
1963      AlertMsg = %Trim(AlertMsg) + 'Required: '
1964                + %Char(CustRqDt) + CrLf;
1965      AlertMsg = %Trim(AlertMsg) + 'Description: '
1966                + %Trim(CustDesc) + %Trim(CustCmt) +
1967                CrLf;
1968      AlertMsg = %Trim(AlertMsg) + 'Qty: ' +
1969                %Char(CustQOrd) + CrLf;
1970      AlertMsg = %Trim(AlertMsg) + 'Price: ' +
1971      - C14 %Char(CustSlPr) + CrLf +
1972      + C14 %Char(CustSlPr) + CrLf ;
1973      AlertMsg = %Trim(AlertMsg) + 'Extended Price: ' +
1974                %Char(CustQOrd * CustSlPr);
1975
1976      Exec SQL
1977          Fetch Next from Custom Into :CustOrdLn,
1978                                     :CustRqDt, :CustDesc, :CustCmt,
1979                                     :CustQOrd, :CustSlPr;
1980
1981      EndDo;
1982
1983      Exec SQL
1984          Close Custom;
1985
1986      Exsr Notify;
1987
1988      EndSr;
1989      //-----*
1990
1991
1992      //-----*
1993      /* exitProgram archives the order processed and terminates the  *
1994      /* program upon completion                                         *
1995      //-----*
1996      BegSr exitProgram;
1997
1998      // Archive the order file...
1999
2000      Cmd = 'CPY OBJ(' + '''' + %Trim(filename) + '''' + ') ' +
2001            'TODIR(' + '''' + '/orders/archive inbound/' +
2002            %Trim(XMLRetailer) + '/orders' + '''' + ') ' +
2003            'REPLACE(*YES) OWNER(*NEW)';
2004      CmdLen = %Len(%Trim(Cmd));

```

```

2005         ExecCmd (Cmd:CmdLen);
2006
2007
2008         // RMVLNK taken out because the CL will handle this...
2009
2010         Cmd = 'RMVLNK OBJLNK(' + ''' + %Trim(filename) + ''' + ')';
2011         CmdLen = %Len(%Trim(Cmd));
2012         ExecCmd (Cmd:CmdLen);
2013
2014     EndSr;
2015     //-----*
2016
2017
2018
2019     // -----//
2020     // NxtHDOrd subroutine will pull the last order number Harris //
2021     // used and update it for the new cXML order. It will also //
2022     // write to the XML cross reference file of orders. //
2023     // -----//
2024     BegSR NxtHDOrd;
2025
2026     Exec SQL
2027         Select CELORD into :LastOrder from OECtrl;
2028
2029     LastOrder += 1;
2030
2031     Exec SQL
2032         Update OECTRL set CELORD = :LastOrder;
2033
2034     CurDate = %Date();
2035     CurTime = %Time();
2036
2037     Exec SQL
2038     Insert into XMLORDS (XCCOORD, XHDSORD, XPROCISO, XPROCHMS)
2039         Values (:XMLOrderID, :LastOrder, :CurDate, :CurTime);
2040
2041     // Calculate the required date for the order
2042     TempDate = %Date() + %Days(1);
2043
2044     // *Mon = 0 ... Sun = 6
2045     Day_of_Week = %rem(%diff(%date() +
2046         %Days(1):d'0001-01-01':*days):7);
2047     If Day_of_Week = 5; //Saturday
2048         TempDate = %Date() + %Days(3);
2049     EndIf;
2050     If Day_of_Week = 6; // Sunday
2051         TempDate = %Date() + %Days(2);
2052     EndIf;
2053
2054     LastDtldate = TempDate;
2055     // Seed header date to zero; CalcReqD sets it to the earliest
2056     // line required date. Empty orders fall back after the loop.
2057     H1RqDt = 0;
2058
2059     EndSR;

```



```

2060 // -----//
2061
2062 // -----//
2063 // CalcReqD subroutine will calculate the required date for //
2064 // individual detail line item based on Bill's criteria of //
2065 // the number of days to add based on the item ordered. It //
2066 // will also take into account any holidays and weekends and //
2067 // not include those in the data calculations. //
2068 // -----//
2069 BegSR CalcReqD;
2070
2071 TodayISO = %Date();
2072 TempDate = TodayISO;
2073
2074 // Add AddDays *business* days: step one calendar day at a
2075 // time, skipping Saturdays, Sundays, and holidays until
2076 // AddDays working days have been counted. The order date
2077 // itself is not counted (Tue + 1 business day = Wed).
2078 DoW AddDays > 0;
2079
2080 TempDate = TempDate + %Days(1);
2081
2082 // Mon = 0 ... Sun = 6
2083 Day_Of_Week = %rem(%diff(TempDate:
2084 d'0001-01-01':*days):7);
2085 If Day_Of_Week = 5 Or Day_Of_Week = 6; // Sat / Sun
2086 Iter;
2087 EndIf;
2088
2089 // Company holiday?
2090 Exec SQL
2091 Select Count(*) Into :DaysOff
2092 From PROITRG/UDCDetail
2093 Where Date(UDCIndexed) = :TempDate
2094 And UDCSystemD = 'HOLIDAY';
2095 If DaysOff > 0;
2096 Iter;
2097 EndIf;
2098
2099 AddDays -= 1; // a real working day
2100
2101 EndDo;
2102
2103 // TempDate builds the actual O1RqDt...
2104 O1RqDt = %Dec(%Char(TempDate:*CYMD0):7:0);
2105
2106 // Header required date = earliest of the line required dates
2107 If H1RqDt = 0 Or O1RqDt < H1RqDt;
2108 H1RqDt = O1RqDt;
2109 EndIf;
2110
2111 EndSR;
2112
2113
2114

```

```

2115 // -----//
2116 // ClearOrd subroutine will clear all the processing for this //
2117 // PO# since the order is rejected due to a bad item. //
2118 // -----//
2119 BegSR ClearOrd;
2120
2121 Exec SQL
2122     Delete From OEHDWK Where H1ORRF = :HWPO#;
2123
2124 Exec SQL
2125     Delete From OEDTWK Where O1OCTL = :LastOrder;
2126
2127 // Notify users of the cancellation....
2128 Exsr Notify;
2129
2130 EndSR;
2131 // -----//
2132
2133
2134
2135 // -----//
2136 // Notify subroutine will send notification of rejection for //
2137 // the order to the appropriate individuals. //
2138 // -----//
2139 BegSR Notify;
2140
2141 Exec SQL
2142     Declare Emails Cursor for
2143         Select UDCDESC1 from PROITRG/UDCDetail
2144             Where UDCSYSTEMD = 'EMAIL' and UDCCODED = :Alert;
2145
2146 Exec SQL
2147     Open Emails;
2148
2149 Exec SQL
2150     Fetch Next from Emails into :Distribution;
2151
2152 DoW SQLCod = 0;
2153
2154 // Actual email routine
2155 from = 'hdfiles@screen-graphics.com';
2156 to = Distribution;
2157 replyTo = *Blanks;
2158 subject = AlertSubj;
2159 text = AlertMsg;
2160
2161 Monitor;
2162     email = RPGMail_new();
2163     RPGMail_addAddress(email:'FROM':from: ' ');
2164     RPGMail_addAddress(email:'TO':to: ' ');
2165     RPGMail_setSubject(email:subject);
2166     RPGMail_setBodyText(email: text: 'text/plain');
2167     RPGMail_setSMTPHost(email: 'MAILGATE');
2168     RPGMail_setSMTPPort(email: '25');
2169     RPGMail_send(email);

```

```

2170      On-error;
2171      error = Error_catch();
2172      // Don't swallow the failure - log it to the job log.
2173      LogTxt = 'XML850R_R1: e-mail send failed for ' + %Trim(to) +
2174              ' (alert ' + %Trim(Alert) + '). ' +
2175              %Trim(error.text);
2176      SndPgmMsg( *blanks : *blanks : LogTxt
2177                : %Len(%Trim(LogTxt)) : '*DIAG' : '*PGMBDY' : 1
2178                : MsgKey : ApiErr );
2179      EndMon;
2180
2181      Exec SQL
2182          Fetch Next from Emails into :Distribution;
2183
2184      EndDo;
2185
2186      Exec SQL
2187          Close Emails;
2188
2189 +      C20
2190 +      C20      // C20 - reset so the next alert cannot inherit this
2191 +      C20      //          one's subject or body text.
2192 +      C20      Clear Alert;
2193 +      C20      Clear AlertSubj;
2194 +      C20      Clear AlertMsg;
2195      EndSR;
2196      // -----//

```